Signal iOS 8.35.1 (build 1964)

Source-to-binary equivalence of the App Store build

Prepared by commandersaki with Claude (Opus 5.5, Anthropic) in Claude Code · 7 November 2026

Summary. The App Store build of Signal iOS 8.35.1 (1964) was decrypted. Then the public source at tag `8.35.1.1964` (commit `4f019b8`) was built with the same Xcode (27.1, build 27A9275). For all five binaries compiled from Signal's own source (`Signal`, `SignalServiceKit`, `SignalUI`, `SignalNSE`, `SignalShareExtension`), **the compiled code section (`__TEXT,__text`) is byte-identical to the shipped app**. Every other differing byte in those binaries was traced to a specific harmless cause. With the code signature removed, the shipped main executable differs from a local build by **one byte**, the signature-size field. 18 of the 19 bundled third-party frameworks are identical apart from that same field. The remaining one, WebRTC, matches once the App Store's architecture thinning is accounted for. Two planted-backdoor builds were both detected, including a one-line change that adds no strings or symbols.

This shows that the shipped binary is what the public source produces. It does not show that the public source itself is trustworthy (see Section 7).

Contents

1 Original request 1
2 Inputs and their provenance 2
3 Methodology 3
4 Tooling written for this project 4
5 Results 5
6 Comparison with Signal Android's reproducible builds 7
7 Limitations 8
8 What the model can conclude 8
9 Cost and effort 9
10 Reproducing these results 9
11 Changelog 10

1 Original request

The request that started this work, verbatim:

```
This is the signal ios git clone, it is open source.

I have placed in the signal_dec directory the signal ipa decrypted and unzipped.

It is known that with iOS you cannot do reproducible builds.

I would like to try to validate that the source repository when compiled has a high
equivalence to the built iphone app.

Would you be able to generate the right build, and then decompile the iphone app and marry
the two to see if there's high equivalence. This would help minimise doubt that there is a
backdoor placed by signal and that signal is operating as intended.
```

`Let me know what tools you need me to install first via brew.`

A follow-up asked for a validation harness, so that the results could be checked instead of taken on trust: *"Is there a validation harness you can do to make sure you're not hallucinating on results."* That request shaped the methodology. Every figure in this report comes from a script's output in `verify/out/`, never from the model's own judgement, and each analysis layer was tested against controls designed to fail.

2 Inputs and their provenance

2.1 The shipped app

Item	Value
App	Signal 8.35.1, `CFBundleVersion` 1964
Built by Signal with	Xcode 27.1 (27A9275), iOS SDK 27.1, on Xcode Cloud (build paths `/Volumes/workspace/repository/...`)
Embedded build metadata	`BuildDetails.SignalCommit` = `4f019b8ba2d10b924d68b4b294bfceb05bde2aa4 Bump version to 8.35.1`, built 30 Oct 2026 17:11 UTC
Encrypted IPA	`signal.ipa` as downloaded from the App Store (SHA-256 withheld: the file embeds a FairPlay licence tied to the downloading account)
Decrypted IPA	`signal.decrypted.ipa` (SHA-256 withheld for the same reason; per-binary hashes are in `verify/out/provenance_ipa.txt`)
Analysed copy	`signal_dec/` (the decrypted IPA, unzipped)

2.2 Decryption with unfairplay

App Store binaries are FairPlay-encrypted, so the IPA was decrypted with **unfairplay** (github.com/DrWhax/unfairplay). The analysis was given the **unzipped** decrypted app (`signal_dec/`), not the decrypted IPA itself. Because the decryption step sits between Apple's file and everything else, it was checked independently with `verify/validate_decryption.py`. The script reads both IPAs directly as zip archives:

Check	Result
`.app` contents of the decrypted IPA vs `signal_dec/`	✓ byte-identical (0 mismatching members)
Mach-O binaries carrying FairPlay encryption	22 of 22; `cryptid` 1 → 0
Bytes changed by decryption **outside** the encrypted ranges	✓ 0 (across all 22 binaries)
Code-signature page hashes (SHA-256) on the **encrypted** IPA	✓ 27,120 / 27,120 match: the input is intact as signed
Decrypted pages that mismatch outside the encrypted range	✓ exactly one per binary, the page holding the `cryptid` field (22/22)
Non-binary files	382 identical; `iTunesMetadata.plist` (purchase metadata, outside the app bundle) differs; 112 `SC_Info` licence files removed
unfairplay's own manifest	`complete: true`, `plaintext_verified: true` against pre-encryption hashes

So decryption changed only the encrypted ranges and the `cryptid` flag. Everything else in the analysed copy is exactly as Apple signed it. One caveat remains. The code signature's page hashes cover the **encrypted** bytes, so they cannot by themselves prove the decrypted plaintext is correct. That proof

comes from unfairplay's pre-encryption-hash check and, independently, from the result of this whole report: decrypted code that was wrong would not be byte-identical to code compiled from source.

3 Methodology

3.1 Reproducing Signal's build

The goal was to match Signal's Xcode Cloud release build as closely as possible:

- **Source:** a fresh clone from GitHub at tag `8.35.1.1964`. It resolves to the same commit, `4f019b8…`, as the commit the shipped app records.
- **Path:** a sparse APFS disk image mounted as `/Volumes/workspace`, so the source sits at `/Volumes/workspace/repository`, the same path as on Xcode Cloud. Build paths embedded in binaries therefore match.
- **Dependencies:** `make dependencies`, as Signal's `ci_scripts/ci_post_clone.sh` runs it. This gives the public `Signal-Pods` submodule at `6703bb87`, plus libsignal `v0.105.0` and RingRTC `v2.73.0`/WebRTC `8037a` prebuilt binaries, checked against the SHA-256 checksums pinned in the `Podfile`.
- **Toolchain:** Xcode 27.1 (27A9275), matching Signal's exact build. The Metal toolchain (27A266a) was a separate download.
- **Build:** `xcodebuild archive -scheme Signal -destination generic/platform=iOS` with the scheme's own Archive configuration, **App Store Release**. Code signing was turned off (`CODE_SIGNING_ALLOWED=NO`), since Signal's certificates are not available. The `production` feature-flag level is already committed at this tag for non-DEBUG builds.
- A clean build took about 6 minutes on this Mac (macOS 26.7, Apple silicon).

3.2 Four builds

Build	Purpose	Source
A	The candidate compared with the App Store app	tag, unmodified
B	Noise control: identical source, clean rebuild	tag, unmodified
C	Planted backdoor (noisy): key exfiltration to a URL, key weakening, and a call redirect	tag + `verify/controls/planted_backdoor.patch`
D	Planted backdoor (subtle): one line changed in place, zeroing the last AES key byte; no new strings, symbols or calls, and no line-number shift	tag + `verify/controls/planted_subtle.patch`

3.3 Comparison layers

Each pair (App Store app, build) is compared in layers, running from coarse to fine:

1. **Bundle inventory:** the SHA-256 of every file, grouped by kind (binary, plist, asset catalog, signature, resource).
2. **Binary features:** every string in the C-string, ObjC class/selector/type and Swift reflection sections, plus imported and exported symbols, linked libraries and section sizes, compared as multisets.
3. **Functions (assembly):** each binary's `__text` is split into functions using `LC_FUNCTION_STARTS` and disassembled with Apple's LLVM disassembler. Opcodes, registers and constants are kept exactly. Each **address** is replaced by what it points to: the imported symbol called through a stub, the ObjC selector of an `objc_msgSend` stub, the text of a loaded string, the referenced GOT symbol, or, for calls within the binary, the callee's own fingerprint. A redirected call or a swapped string therefore changes the fingerprint, while a mere shift in layout does not.

4. **Raw bytes:** the code signatures are removed with `codesign --remove-signature`, then every differing byte is assigned to the section or load command it falls in, so no difference goes unaccounted for. Raw `__text` hashes are taken with Apple's `segedit` and `shasum`, independently of any script written for this project.

What was expected at the outset. Before any build was made, the model told the user that this analysis "won't produce a bit-for-bit match. Signing, build paths, timestamps, link order and optimiser differences all get in the way, so the result is 'highly equivalent, with these differences explained.'" The original request also asked only for "high equivalence". On that expectation, nearly every byte of `__text` would differ because addresses shift, and only an address-independent comparison could show the code was the same. That is why the function layer was built, and why the plan also listed radare2 and Ghidra for diffing functions that did not match.

What happened. It turned out to be more straightforward than that. With the right build parameters (the same Xcode build, the Xcode Cloud build path `/Volumes/workspace/repository` and the scheme's own App Store Release configuration), the compiled code came out byte-identical. Build paths and timestamps did not leak into `__text`, and the only link-order effect was the GOT-slot ordering in the stubs. The raw-byte comparison alone is therefore the primary evidence, and radare2 and Ghidra were not needed. The function layer remains as an independent second method, a way to make sense of the GOT-ordering differences in the stubs, and a fallback for future versions that do not reproduce exactly.

3.4 Guarding against a wrong result

Since a language model wrote both the harness and this report, the methodology assumes the model could be wrong and checks for it:

- **Results come only from scripts.** Figures in this report are copied from files in `verify/out/`, and `verify/compare_build.sh` re-creates them.
- **Independent cross-checks:** the string parser was checked against Apple's `segedit` (agreement on all ≈52,000 strings), and function boundaries against Apple's `dyld_info` (297/297 on the NSE). The most important result, byte-identical `__text`, relies only on Apple's tools.
- **Controls designed to fail:** an identical copy of the app (must match 100%), a tampered copy (must flag exactly the change), a rebuild of identical source (measures noise), and two planted backdoors (must be detected).
- **Determinism:** fingerprinting the same file twice gives identical output.

4 Tooling written for this project

All scripts are in `verify/`, use only the Python standard library and Apple's command-line tools, and run with `python3 -I`.

Script	Role
`provenance.sh`	Records tool versions, the source and Pods commits, and the SHA-256 of every shipped binary
`inventory.py`, `compare_inventory.py`	Per-file hash inventory of an `.app` and its diff
`macho_features.py`	Own Mach-O parser for string sections; Apple's `nm` / `otool` for symbols and libraries
`crosscheck_strings.py`	Checks the parser against Apple's `segedit`; refuses to continue on any disagreement
`compare_features.py`	Multiset comparison of features, listing every differing item
`functions.py`	Function splitting, disassembly, address-to-target resolution (via `dyld_info -fixups`) and two-pass fingerprints
`compare_functions.py`	Function matching by fingerprint; unmatched functions on both sides

`bytediff.py`	Removes signatures and assigns every differing byte to a section or load command
`pngpixels.py`	Decodes Apple CgBI PNGs and compares their pixels
`validate_decryption.py`	Checks a decrypted IPA against the encrypted original
`compare_build.sh`, `summarise.py`	Run every layer for a pair and summarise it

Defects found and fixed during the work. Reporting these is part of the validation:

- The first build used `-configuration Release`, which does not exist in Signal's project. It was replaced with the scheme's **App Store Release**.
- The first string cross-check misread `otool` output and flagged every section. It was switched to `segedit`.
- A parser crash on register-plus-register `add` instructions.
- Export names containing spaces (ObjC methods such as `+[OWSAES256Key supportsSecureCoding]`) were split in half. Fixing this changed no result, because both sides had been split the same way.
- The assumption that code-signature hashes cover the plaintext was wrong. They cover the encrypted bytes, and the decryption check was corrected to match.
- The function layer still produces false positives when a binary's layout shifts (Section 7).

5 Results

5.1 First-party binaries: App Store app vs build A

Binary	Functions	Instructions	`__text` bytes	Feature layers
Signal	50,566 / 50,566	2,652,940	✓ identical	✓ all identical
SignalServiceKit	82,839 / 82,839	4,267,966	✓ identical	✓ all identical
SignalUI	17,175 / 17,175	735,432	✓ identical	✓ all identical
SignalNSE	297 / 297	9,366	✓ identical	✓ all identical
SignalShareExtension	647 / 647	23,419	✓ identical	✓ all identical
Total	**151,524**	**7,689,123**		

"Feature layers" covers, for example in `Signal`: 16,273 C strings, 1,060 ObjC class names, 5,746 selectors, 9,695 Swift field names, 8,209 imports and 100 linked libraries. Every one of these is identical in both builds, with no additions and no removals.

5.2 Accounting for every differing byte

With signatures removed, every pair of binaries has exactly the same size. The bytes that still differ are:

Binary	Bytes	Cause
SignalServiceKit SignalNSE SignalShareExtension	1–2	The `__LINKEDIT` vmsize. It was sized for Apple's signature (for example, 4,209,328 B of link data + 497,408 B of signature, rounded up to 16 KB = `0x480000`). Ours is sized for no signature (`0x404000`). This is caused by signing alone.
Signal	1,710	The same vmsize byte, plus `LC_UUID` (16 B), plus 1,694 bytes in `__stubs` / `__objc_stubs`. The linker made two GOT slots, both bound to `libobjc/_objc_msgSend` in **both** builds (checked with `dyld_info`), and pointed the stubs at them in the opposite order. The UUID depends on these bytes.
SignalUI	1,182	The same pattern: two `_objc_msgSend` slots swapped, 1,165 stub bytes, plus the UUID

The noise control confirms this. In build B, the GOT-slot order happened to match Apple's for `Signal`: **IPA `Signal` vs build B `Signal` differs by exactly 1 byte (the signature-size field)**. Meanwhile `SignalServiceKit` flipped order in B, and builds A and B differ from each other by the same stub pattern. The ordering is therefore linker nondeterminism between builds of identical source, not something added to the App Store build.

5.3 Third-party frameworks

Frameworks	Result after removing signatures
LibSignalClient, SignalRingRTC, SQLCipher, MobileCoin, LibMobileCoin, GRDB, SwiftProtobuf, CocoaLumberjack, Logging, BonMot, Lottie, PureLayout, SDWebImage, SDWebImageWebPCoder, libPhoneNumber, libwebp	✓ identical except the signature-size byte (1–2 bytes)
WebRTC	✓ identical except the signature-size byte, after `lipo -thin arm64`. Signal's prebuilt is a single-architecture "fat" binary, which App Store thinning unwraps. Both have UUID `4C4C44B7-5555-3144-A187-CBA7B54C9452`, matching the RingRTC prebuild Signal published

LibSignalClient (the Signal Protocol implementation), RingRTC/WebRTC and MobileCoin come as **prebuilt binaries** pinned by checksum. This shows the App Store app contains exactly those published prebuilds. It does not show the prebuilds come from their own source (Section 7).

5.4 Non-code files

- **Plists (33 differ):** `BuildMachineOSBuild` (Xcode Cloud `26A434` vs local `25G229`), `CFBundleVersion` (`1964` vs `0`; Xcode Cloud sets it at export), `BuildDetails` date and timestamp, and `DTAppStoreToolsBuild` (added by App Store processing). No other keys differ.
- **Asset catalogs:** identical rendition sets (2,891 and 2,529 renditions), produced by the same Xcode build. Only the 73 app-icon renditions differ. Their names include a random UUID, process ID and timestamp from `actool`, and the two loose icon PNGs differ by at most 1/255 in one channel, in under 1% of pixels (icon-resizing rounding). Icons are images with no executable content.
- **Code signatures:** the 62 `_CodeSignature` files exist only in the signed App Store app.
- Every other file is byte-identical (280 of 339 shared files are identical; each of the remaining 59 is explained above).

5.5 Controls

Control	Outcome	Pass
Identical copy	401/401 files; 50,566/50,566 functions	✓
Tampered copy (1 byte appended, 1 file added)	Flags exactly that file and the added file	✓
Build B, identical source	`__text` identical in all 5 binaries; only GOT-order noise, which changes between builds	✓
Build C: key exfiltration	New strings `https://planted-control.invalid/`, `collect planted-backdoor-control`; new selector `sharedSession`; changed `__text`	✓

Build C: call redirect in `Signal`	Caught as a changed caller (`0x10048d5ec`). Swift inlined the new callee, so it was a body change, not a pure target swap	✓
Build D: one-line key weakening	`SignalServiceKit` `__text` hash changed, the other 4 binaries unchanged; **no** string, import or metadata difference (as expected); the changed function is identified and readable: `count - 1`, a `Data` subscript set to `0`, then `_crypt(op: kCCEncrypt)`	✓

In build C, most of the 652 flagged `SignalServiceKit` functions changed for a genuine reason: inserting 14 lines shifts the source line numbers compiled into assertions (`OWSAssertionError` records `#line`). 44 of the 51 changed constants are exactly +14. The layers are therefore **more** sensitive than strictly necessary.

6 Comparison with Signal Android's reproducible builds

Signal Android publishes a reproducible-build procedure (`reproducible-builds/README.md`). Its build environment is a Docker image from the repository's `Dockerfile`, built from the release's git tag. The user builds the Play bundle in Docker, uses `bundletool` to generate the APKs the Play Store would install on their device, pulls the installed APKs with `adb`, and compares them with `apkdiff.py`. That tool ignores "signing information" and "extra harmless metadata added by the Play Store". Success means every pair (base, ABI and density splits) prints "APKs match!".

	Signal Android (official)	**This iOS analysis**
Build environment	Docker image defined in the repo, maintained by Signal	Xcode version pinned by `.xcode-version`; Xcode Cloud path recreated by hand; not containerised (macOS/Xcode cannot be)
Getting the shipped artefact	`adb pull` of the installed APKs; no decryption needed	FairPlay-encrypted IPA, decrypted with a third-party tool (unfairplay), then validated
Comparison	`apkdiff.py`: archive contents compared, signatures and Play Store metadata ignored	Custom multi-layer harness; signature removed; every remaining byte accounted for
Result format	Binary verdict: "APKs match!"	Byte-identical code, plus an itemised list of explained non-code differences
Who can do it	Anyone with Docker; documented and supported	Needs a Mac, the exact Xcode build, and an iOS decryption step; not supported by Signal
Re-verification	Every release, by anyone	This one release; scripts in `verify/` can be re-run for others

The two reach a comparable standard by different routes. Android's procedure is official, repeatable and easy for anyone to follow, but its verdict is binary and relies on `apkdiff.py`'s list of ignored differences. This iOS analysis is improvised and much harder to repeat. However, for the code it arrived at the same strong result, byte-identical compiled code, and it explains every byte that is not identical instead of leaving it out. "iOS builds cannot be reproduced" turned out not to apply to the code: with the same Xcode build and build path, Signal's iOS code reproduced exactly. What could not be reproduced is Apple's signature, FairPlay encryption and App Store post-processing, and those were handled by removing or reversing them and checking the result.

7 Limitations

1. **Trust in the source is a separate question.** This work shows the App Store binary equals what the public source produces. A backdoor **committed to the public repository** would reproduce exactly and pass every check here. Ruling that out needs a source review, focusing on code paths that handle keys, plaintext and networking.
2. **Prebuilt dependencies are checked against their published binaries, not their source.** LibSignalClient (`v0.105.0`), RingRTC (`v2.73.0`) and WebRTC (`8037a`) are downloaded prebuilt and checked against the checksums in the `Podfile`. Rebuilding them from Rust/C++ source and comparing would close this gap, and is a larger separate task.
3. **Private Pods repository.** `Scripts/setup_private_pods` can switch the build to `Signal-Pods-Private`. Whether Signal's release builds do so is an Xcode Cloud setting that can't be inspected. For this release it makes no difference: every framework built from Pods matched the public Pods exactly.
4. **One release, one copy.** The result covers 8.35.1 (1964), this downloaded copy and one device class (App Store thinning). Other versions, other thinned variants, or another user's download were not checked.
5. **Outside the IPA.** What iOS does at install or run time, and code loaded at run time (if any), are outside what a static comparison can see. Server-side behaviour is out of scope.
6. **Decryption trust.** The decrypted plaintext relies on a third-party tool. It is backed by that tool's plaintext-hash verification and, more strongly, by the plaintext being byte-identical to independently compiled code, which a faulty decryption could not plausibly produce.
7. **Function-layer false positives.** When layout shifts (builds C and D), the function layer flags extra functions. These come from LLVM's machine outliner, which passes page addresses in registers between functions; the per-function tracker cannot resolve them. In build D, 13 of 14 directly changed functions were such false positives, each a single 16-byte constant load shifted by `+0x140`. This affects **locating** changes in a shifted binary, not **detecting** them, and it cannot affect the main result: the A comparison had no shift, and its match is backed by byte-identical `__text`.
8. **Not every flagged item was traced.** In build C, 7 of the 51 changed constants and one register-allocation change in build D (`0xbed94c`) were not individually traced to a source line.

8 What the model can conclude

I'm Claude, the model that did this analysis and wrote this report, and I want to state my conclusions carefully. I wrote the tools that produced the evidence, so the strongest claims below rest on checks that don't depend on them: Apple's `segedit` / `shasum` hashes and the planted controls.

Supported by the evidence:

- The compiled code shipped in Signal iOS 8.35.1 (1964) for Signal's own five binaries is **byte-for-byte** the code produced by compiling the public tag `8.35.1.1964` with Xcode 27.1 (27A9275).
- Nothing was added, removed or altered in those binaries apart from a code signature, its size field, and a linker GOT-slot ordering that varies between identical builds.
- The bundled third-party frameworks are exactly the versions the public repository pins.
- The harness is capable of catching a hidden change: a single-line key weakening, with no new strings or symbols, was detected and traced to the function that contains it.

Therefore, for this release, a backdoor **added by Signal (or anyone in its build pipeline) after the public source** is ruled out to a very high degree of confidence. The App Store app runs the code in the public repository.

Not supported: this says nothing about whether the public source contains a backdoor or a vulnerability, nothing about server behaviour, and nothing about other releases. "Operating as intended" holds in the narrow sense that the shipped app matches its public source. Whether that source does what Signal says it does is a separate review.

9 Cost and effort

The figures below were reported by Claude Code's `/cost` command at the end of the session, before this section was added. They cover the whole conversation: planning, harness development, the four builds, the analysis and the report.

Measure	Value
Total cost	**$9.22** (USD)
Main model, `claude-opus-5-5`	1.6k input · 139.5k output · 22.0M cache read · 251.6k cache write tokens ($9.22)
Helper model, `claude-haiku-5-5`	10.0k input · 1.7k output tokens ($0.0019)
Prompt cache	133 requests; 99% of input tokens served from cache; no misses
API time	25 min 20 s
Wall-clock time	1 h 23 min 46 s (mostly Xcode builds and user actions such as installing Xcode)
Code written	1,592 lines added, 147 removed (the `verify/` harness and this report)

Most of the cost is in the cache reads. Each turn sends the whole conversation back as context, and long tool outputs such as build logs and comparison tables add to that. The Xcode builds (about 6 minutes each, four in total, plus two failed attempts) used local compute only and no tokens.

10 Reproducing these results

```
# 1. Source at the Xcode Cloud path
hdiutil create -size 150g -type SPARSE -fs APFS -volname workspace workspace.sparseimage
hdiutil attach workspace.sparseimage -nobrowse
git clone https://github.com/signalapp/Signal-iOS.git /Volumes/workspace/repository
cd /Volumes/workspace/repository && git checkout 8.35.1.1964
Scripts/check_xcode_version.py && make dependencies
xcodebuild -downloadComponent MetalToolchain

# 2. Build (scheme's App Store Release configuration, unsigned)
xcodebuild archive -workspace Signal.xcworkspace -scheme Signal \
  -destination 'generic/platform=iOS' -derivedDataPath /Volumes/workspace/DerivedData \
  -archivePath /Volumes/workspace/build-A.xcarchive \
  CODE_SIGNING_ALLOWED=NO CODE_SIGNING_REQUIRED=NO CODE_SIGN_IDENTITY=""

# 3. Compare (from this repository's root, with the decrypted app unzipped to signal_dec/)
verify/compare_build.sh signal_dec/Payload/Signal.app \
  /Volumes/workspace/build-A.xcarchive/Products/Applications/Signal.app verify/out/
ipa_vs_A
python3 -I verify/summarise.py verify/out/ipa_vs_A
python3 -I verify/bytediff.py <ipa binary> <built binary> out.json
python3 -I verify/validate_decryption.py signal.ipa signal.decrypted.ipa out.json
signal_dec/Payload/Signal.app
```

Outputs: `verify/out/ipa_vs_A/` (main comparison and byte accounting), `verify/out/controls/` (identity, tamper, built-twice, planted C and D), `verify/out/decryption_validation.json`, `verify/out/provenance_ipa.txt`. Controls: `verify/controls/*.patch`.

11 Changelog

Date	Change
7 November 2026	First published version.
7 November 2026	**Added** to Section 3.3: what was expected at the outset (no bit-for-bit match, hence the function layer), and what happened (the right build parameters produced byte-identical code, making the raw-byte comparison the primary evidence). **Added** this changelog. Nothing deleted.